ICT162

Timed Online Assignment - January Semester 2023

Object Oriented Programming

Thursday, 25 May 2023 **04:00 pm - 06:00 pm**

Time allowed: 2 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **4** questions and comprises **14** pages (including cover page).

2. You must answer **4** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. **Images of handwritten answers will not be marked**.

6. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

ICT162 Copyright © 2023 Singapore University of Social Sciences (SUSS)

(Full marks: 100)

Question 1

Study the program given in Figure Q1 and answer the questions that follow:

```
from abc import ABC, abstractmethod

class Employee(ABC):
    def __init__(self, employeeId, name, workFromHome):
        self._employeeId = employeeId
        self._name = name
        self._workFromHome = workFromHome
        self._leaveBalance = 0

    def workFromHome(self, atHome):
        self._workFromHome = atHome

    @property
    def workFromHome(self):
        return self._workFromHome

    @abstractmethod
    def getLeaveEntitlement(self):
       pass

class PartTimeEmployee(Employee):
    _LEAVE_ENTITLEMENT = {15: 5, 30: 10, 99: 12}

    def __init__(self, employeeId, name, workFromHome, hoursPerWeek):
        super().__init__(employeeId, name, workFromHome)
        self._hoursPerWeek = hoursPerWeek
        self._leaveBalance = self. getPartTimeLeaveEntitlement()

    def getPartTimeLeaveEntitlement(self):
        for k, v in PartTimeEmployee._LEAVE_ENTITLEMENT.items():
            if self._hoursPerWeek <= k:
                return v
        else:
            return v

class FullTimeEmployee(Employee):
    _LEAVE_ENTITLEMENT = {4: 22, 3: 20, 2: 18, 1: 16}

    def __init__(self, employeeId, name, workFromHome, grade):
        super().__init__(employeeId, name, workFromHome)
        self._grade = grade
        self._leaveBalance = self.getLeaveEntitlement()

def getLeaveEntitlement(self):
    minLeave = min([leave for leave in
                            FullTimeEmployee._LEAVE_ENTITLEMENT.values()])
        return FullTimeEmployee._LEAVE_ENTITLEMENT.get(self._grade,
minLeave)

class Manager(FullTimeEmployee):
_LEAVE_ENTITLEMENT = {4: 25}
```

Figure Q1

Question 1a

Explain what the following statements are doing and what is the output.

```
f1 = FullTimeEmployee(102, "Jim", True, 4)
              print( f1.getLeaveEntitlement() )
```

(3 marks)

Question 1b

Demonstrate what error will be encountered when executing the statement below.

```
p1 = PartTimeEmployee(992, "Terry", False, 30)
```

Make the necessary corrections to class PartTimeEmployee, so that the statement will execute.

(4 marks)

Question 1c

State whether there is an error when executing the following statements below.

```
f2 = FullTimeEmployee(104, "Jack", False, 3)
f2.workFromHome = True
print( f2.workFromHome )
```

- If there is error, specify the statement line number(s) in Figure Q1 that causes the error. Explain why the error occurs and make correction(s) to the part if it is possible to correct the error.
- If there is no error, explain what the statements are doing.

(4 marks)

Question 1d

The statements below will execute successfully, and output is shown.

```
m1 = Manager(106, "Tom", False, 4)
print( m1.getLeaveEntitlement() )

output:  22
```

However, the output is not correct as Manager should be given 25 days of leave.

- Identify the statement line number(s) in Figure Q1 that causes the wrong leave entitlement.
- Explain why the error occurs and make correction(s) to the identified statement(s) to compute the correct leave entitlement.

(4 marks)

Question 1e

Describe the terminology polymorphism, and provide an example using objects created with the classes in Figure Q1.

- Indicate which statement in your code exhibits polymorphism.

(5 marks)

ICT162 Copyright © 2023 Singapore University of Social Sciences (SUSS)

Question 2

The class diagram in Figure Q2 shows specifications of the food items sold in a café. The Deli class and the ColdDeli class are described in Table Q2(a) and Table Q2(b) respectively.

Demonstrate your understanding of how class hierarchy and association are used to organize information, by constructing the class hierarchy and association according to the specifications.

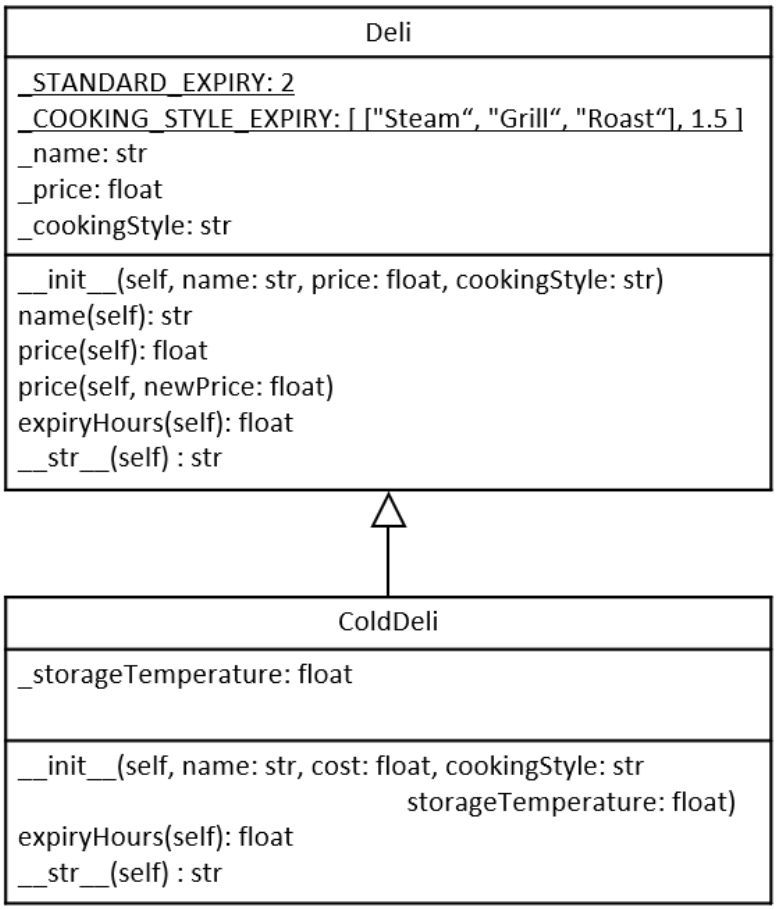


Figure Q2

Question 2a

Write a Python class definition for the Deli class.

The Deli class represents a food item that the café is offering.
• It has the following attributes: name, price and cookingStyle.
• There are two class variables:
  ○ _STANDARD_EXPIRY defines the standard expiry hours of a deli as 2 (hours)
  ○ _COOKING_STYLE_EXPIRY defines a list of cooking styles with expiry hours differing from standard. In Figure Q2, it shows that food item that is prepared via "Steam, "Grill" and Roast" will have expiry hours as 1.5

The Deli class has the following methods:
• Constructor that initialises a new Deli with the parameter name, price and cookingStyle.
• Getter methods for name and price. Use the property decorator.
• Setter for price. Use the setter decorator.
• The method expiryHours checks if cookStyle is one of those in the class variable _COOKING_STYLE_SEXPIRY. If it is, the method returns 1.5 as expiry hours. Otherwise, the standard expiry hours should apply.
• The __str__ method returns a string representation of a Deli object, including the name, price and expiry hours of the Deli objects. This is an example in the suggested format:
 	
`Name: Roast Beef  Price: $3.90`
`Expiring in 1.5 hrs`

Table Q2(a)

(10 marks)

Question 2b

Write a Python class definition for the ColdDeli class.

The ColdDeli class has additional attribute storageTemperature that will help determine how long cold food items can last.
The ColdDeli class has the following methods, in addition to the methods of the Deli class: • Apart from name, price and cookingStyle, constructor also initialises the additional instance variable storageTemperature. • The method expiryHours ignores the cookingStyle, but examines the following conditions to define the expiry hours for ColdDeli: <table><tr><th>Storage Temperature</th><th>Expiry hours</th></tr><tr><td><= 10 °c</td><td>0.5</td></tr><tr><td><= 15 °c</td><td>1.0</td></tr><tr><td><= 20 °c</td><td>1.5</td></tr></table>If the storageTemperature is above 20 °c, the standard expiry hours should apply. • The __str__ method returns a string representation of a ColdDeli object. See example below: Name: Strawberry Pudding Price: $2.00 Expiring in 0.5 hrs Store at 2.0 degree celsius

Table Q2(b)

(6 marks)

Question 2c

Describe what type of overriding you have implemented for expiryHours and __str__ methods in class ColdDeli. What is the OO concept that allows for overriding?

(3 marks)

Question 2d

Write Python statements to invoke the methods you have defined.

(i) Create the appropriate Deli objects using the information below.

Name	Cost	Storage Temperature	Cooking Style
Strawberry Pudding	$2.00	2°c	Whisk
Roast Beef	$3.90	-	Roast
Chicken Pie	$2.80	-	Bake
Chocolate Steamed Cake	$1.50	15°c	Steam

(2 marks)

(ii) Add 20 cents to the price of all Deli objects created in Q2(d)(i).

(2 marks)

(iii) Print the expiry hours for "Chocolate Steamed Cake". Based on your understanding, what should be the output?

(2 marks)

Question 3

The class diagram in Figure Q3 shows DeliSet class and the DeliDispenser class. The classes are described in Table Q3(a) and Table Q3(b) respectively.

Apply the principles of object-oriented programming and develop an application to help Café customer create their own customised Deli set, add, and remove Deli from a Deli Dispenser. The Deli Dispenser contains inventory of Delis and ColdDelis which customers can select to create their own Deli meals. The application handles exceptions from a user defined exception class, DeliSetException, which you will define for this question.

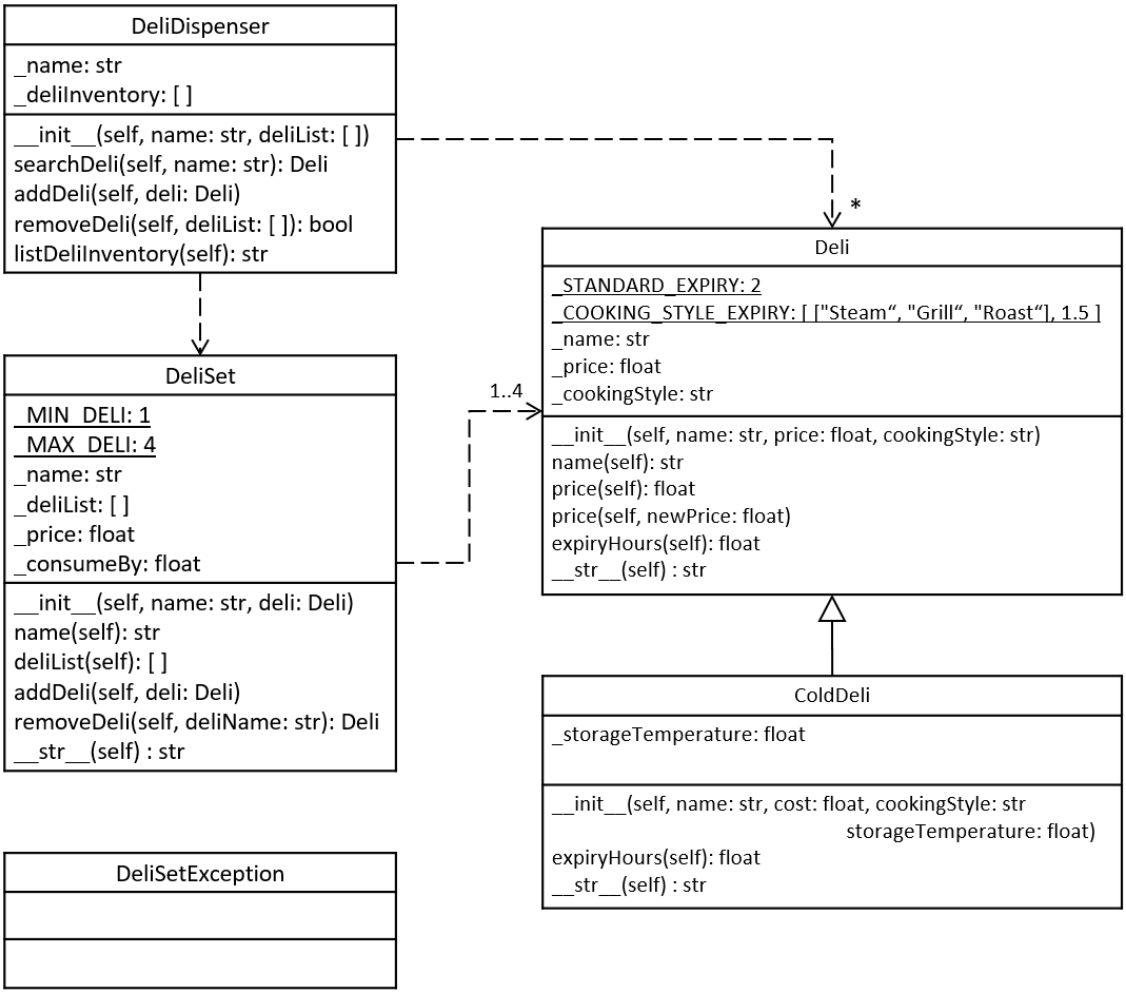


Figure Q3

The DeliSet class represents a customised meal with selected Delis by the customer. It has the following attributes:

- Three class variables:
  - _MIN_DELI and _MAX_DELI represent the minimum and maximum number of delis in a meal set.
  - _MIN_PRICE represents the minimum price the DeliSet must meet.
- _name represents the customer defined name for this meal set.
- _deliList is a list containing the customer selection of Deli objects for this meal set.
- _price represents the final price of this meal set.
- _consumeBy indicates the expiry hours of the meal set.

The DeliSet class has the following methods:

- The constructor performs the following:
  - Initialise _name with the given parameter name.
  - Initialise _deliList as a list with one Deli object, which is the given parameter deli.
  - _price of this meal set is the price of the parameter deli, since it is the only deli in this set.
  - _consumeBy is set to the expiry hours of the parameter deli.
- Getter methods for name and deliList. Use the property decorator.
- addDeli method has a Deli object as parameter and adds it into the _deliList, only if the following conditions met:
  - With this addition, the number of delis should not be more than _MAX_DELI. Otherwise, raise DeliSetException with appropriate message.
  - There is no duplicated Deli within this meal set (i.e., cannot have same name). Otherwise, raise DeliSetException with appropriate message.
  - _price is re-computed by obtaining the sum of all deli's price.
  - _consumeBy is set to the lowest expiry hours of all delis.
- removeDeli method removes from _deliList, the Deli object that matches the parameter deliName. It returns the Deli object only if the following conditions met:
  - With this removal, the number of delis should not be less than _MIN_DELI. Otherwise, raise DeliSetException with appropriate message.
  - There must be a Deli object matching the given deliName. If not, raise DeliSetException with appropriate message.
  - Price and consumeBy is re-computed after each successful removal addition.
- The __str__ method returns a string representation of a DeliSet object, including the name of the Deli objects. This is an example in the suggested format:

```
Deli: My Breakfast      Price: $4.80     Consume in 1.0 hrs

      with Chicken Pie, Strawberry Pudding
```

Table Q3(a)

Question 3a

Write a Python class definition for the exception class DeliSetException that does not have any additional attributes or methods.

(1 mark)

Question 3b

Implement the DeliSet class, as described in Table Q3(a) and Figure Q3.

(15 marks)

Question 3c

Assume the DeliDispenser class described in Table Q3(b) has been written.

The DeliDispenser class has the following attributes: • _name that represents the Café or eatery. • _deliInventory is a list containing all the delis that the Café has. There can be duplicated delis, for example, when there are 2 or more Strawberry Puddings.

The DeliDispenser class has the following methods:
 	
• __init__(self, name, deliInventory) initialises a new DeliSet with the parameters name and a list containing current inventory of delis.
• searchDeli method returns the first Deli object whose name matches the parameter name. If no match, the method returns None.
• addDeli method adds a given Deli object into the inventory.
• Given a list of delis, the method removeDeli will remove each deli from the _deliInventory.
• listDeliInventory returns a string representation the entire inventory of delis. Only the name and price of each deli are to be presented. An example is shown here:
 	
`Name: Strawberry Pudding  Price: $2.00`
`Name: Strawberry Pudding  Price: $2.00`
`Name: Chocolate Steamed Cake  Price: $1.50`
`Name: Smoked Duck Salad  Price: $4.00`
`Name: Roast Beef  Price: $3.90`
`Name: Roast Beef  Price: $3.90`
`Name: Chicken Pie  Price: $2.80`

Table Q3(b)

In addition, an **application** has been partially implemented with a DeliDispenser object created, populated with some deli inventories.

The menu of the application is presented as follows:

```
Menu
1. Create Deli-Set
2. Add Deli to Deli-Set
3. Remove Deli from Deli-Set
4. Show Deli inventory
0. Quit
```

Question 3c(i)

Implement the function createDeliSet(cafe) for Option 1 in the Menu, which allows customers to create their own customised meal. The function is called with parameter cafe, which is a DeliDispenser object.

The createDeliSet function performs the following:

- Prompt the customer for name of the DeliSet.
- List the current inventory of deli for customer to select, until customer keys <Enter> to stop deli selection.
    - Create the DeliSet after customer selected the first deli.
    - Add subsequent selection of deli into the DeliSet. Handle DeliSetException if any.
- Remove the selected delis from the cafe inventory, once the DeliSet is created successfully.
- Advise the customer on the final price and when to consume by printing the DeliSet
- Lastly, the function returns the DeliSet object created.

<u>Sample runs for option 1 with user input bold and underlined:</u>

```
Enter choice: 1
Enter your customised set name: Test Meal
Available Deli:
Name: Strawberry Pudding                    Price: $2.00
Name: Strawberry Pudding                    Price: $2.00
Name: Chocolate Steamed Cake                Price: $1.50
Name: Smoked Duck Salad                     Price: $4.00
Name: Roast Beef                             Price: $3.90
Name: Roast Beef                             Price: $3.90
Name: Chicken Pie                            Price: $2.80

Select Deli by entering the name. <Enter> to stop
Deli <1>: Roast Beeef
No such Deli. Select again.
Deli <1>: Strawberry Pudding
Deli <2>: Chocolate Steamed Cake
Deli <3>: Smoked Duck Salad
Deli <4>: Roast Beef
Deli <5>: Roast Beef
Add Deli failed. Set should not have duplicate Deli
Deli <5>: Chicken Pie
Add Deli failed. Set should not have more than 4 Delis
Deli <5>: <Enter>
Deli: Test Meal          Price: $11.40         Consume in 0.5 hrs
    with Strawberry Pudding, Chocolate Steamed Cake, Smoked Duck Salad, Roast Beef

---
Enter choice: 1
```

```
Enter your customised set name: Happy Meal
Available Deli:
Name: Strawberry Pudding                    Price: $2.00
Name: Strawberry Pudding                    Price: $2.00
Name: Chocolate Steamed Cake                Price: $1.50
Name: Smoked Duck Salad                     Price: $4.00
Name: Roast Beef                             Price: $3.90
Name: Roast Beef                             Price: $3.90
Name: Chicken Pie                            Price: $2.80

Select Deli by entering the name. <Enter> to stop
Deli <1>: Strawberry Pudding
Deli <2>: Roast Beef
Deli <3>: <Enter>
Deli: Happy Meal        Price: $5.90          Consume in 0.5 hrs
          with Strawberry Pudding, Roast Beef
```

(9 marks)

Question 3c(ii)

Apply and implement the function removeDeli(cafe, deliSet) for Option 3 in the Menu, which allows customers to remove Deli object from their own customised meal. The function is called with parameters cafe, a DeliDispenser object and deliSet, a DeliSet object created from Option 1.

The removeDeli function performs the following:

- Display the current DeliSet
- Ask the customer for name of the Deli to remove
- Attempt to remove the Deli from the DeliSet. Handle DeliSetException if any.
- Once the Deli object is removed from the DeliSet, put the Deli object back into the cafe inventory.
- After successful removal, print the DeliSet for customer to verify.

Sample runs for option 3 with user input bold and underlined:

```
Enter choice: 3
Deli: Happy Meal         Price: $5.90           Consume in 0.5 hrs
    with Strawberry Pudding, Roast Beef
Enter Deli name to remove: Chicken Pie
Remove Deli failed. Deli Chicken Pie not found in set

---
Enter choice: 3
Deli: Happy Meal         Price: $5.90           Consume in 0.5 hrs
    with Strawberry Pudding, Roast Beef
Enter Deli name to remove: Roast Beef
Removed successfully... please verify your set
Deli: Happy Meal         Price: $2.00           Consume in 0.5 hrs
    with Strawberry Pudding

---
Enter choice: 3
Deli: Happy Meal         Price: $2.00           Consume in 0.5 hrs
    with Strawberry Pudding
Enter Deli name to remove: Strawberry Pudding
Remove Deli failed. Set should have at least 1 Deli
```

(5 marks)

Question 4

A prototype GUI is developed to allow members in a club to book the use of a gymnasium. The booking is done weekly, and each member can only book once, irrespective of the morning (AM) or afternoon (PM) session. The initial design of the GUI is shown in Figure Q4.

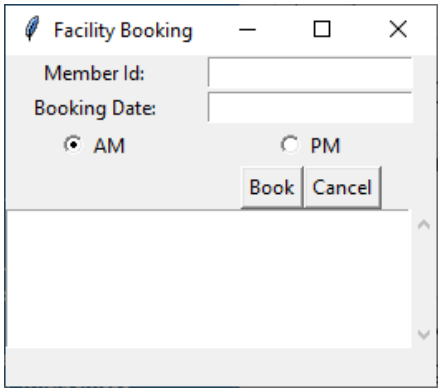


Figure Q4

The following is the initial code for the GUI shown in Figure Q4.

```
1.     from tkinter import *
2.     from tkinter import scrolledtext
3.     from tkinter.scrolledtext import *
4.
5.     class BookingGUI:
6.         def __init__(self):
7.             self._tk = Tk()
8.             self._tk.geometry('330x250')
9.             self._tk.title(Facility Booking')
10.            self.initWidgets()
11.            self._tk.mainloop()
12.
13.        def initWidgets(self):
14.            #create widgets
15.            self._lblId = Label(text='Member Id:')
16.            self._txtId = Entry(width=20)
17.            self._lblDate = Label(text='Booking Date:')
18.            self._txtDate = Entry(width=20)
19.            self._rbtn = IntVar()
20.            self._rbtnAm = Radiobutton(text='AM', value = 0,
21.    variable=self._rbtn)
22.            self._rbtnPm = Radiobutton(text='PM', value = 1,
23.    variable=self._rbtn)
24.            btnFrame = Frame(self._tk)
25.            self._btnBook = Button(btnFrame, text='Book')
26.            self._btnCancel = Button(btnFrame, text='Cancel')
27.            self._sclText = ScrolledText(width=30, height=5)
28.            self._lblId.grid(row=0, column=0)
29.            self._txtId.grid(row=0, column=1)
30.            self._lblDate.grid(row=1, column=0)
31.            self._txtDate.grid(row=1,column=1)
32.            self._rbtnAm.grid(row=2, column=0)
33.            self._rbtnPm.grid(row=2, column = 1)
34.            self._btnBook.grid(row=0, column=0)
35.            self._btnCancel.grid(row=0, column=1)
36.            btnFrame.grid(row=3, column=1)
37.            self._sclText.grid(row=4, column=0, columnspan=2)

       BookingGUI()
```

Question 4a

The user has requested that the radio button for AM and PM should be align at the right as shown in Figure Q4(a).

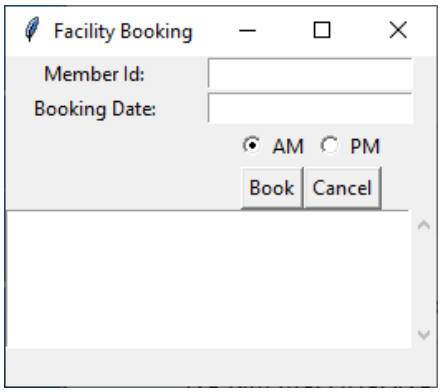


Figure Q4(a)

Based on the code given, write the necessary changes to reflect the new position of the radio buttons.

(6 marks)

Question 4b

Describe or sketch how would the GUI look like with the following statement for the scrolltext.

```
self._sclText.grid(row=4, column=0, columnspan=2)
```

does not have the columspan:

```
self._sclText.grid(row=4, column=0)
```

(2 marks)

Question 4c

For simulation purpose, a few member bookings are to be hardcoded for this booking week. The data structure to use is as follows:

```
{'m1':['1/10','AM'], 'm2':['1/10','AM'], 'm3':['1/10','AM']}
```

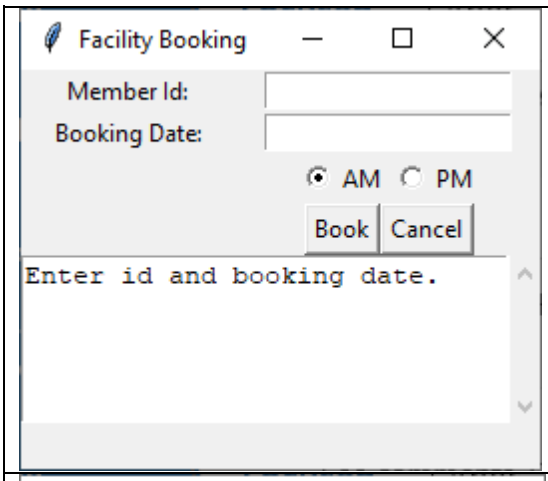
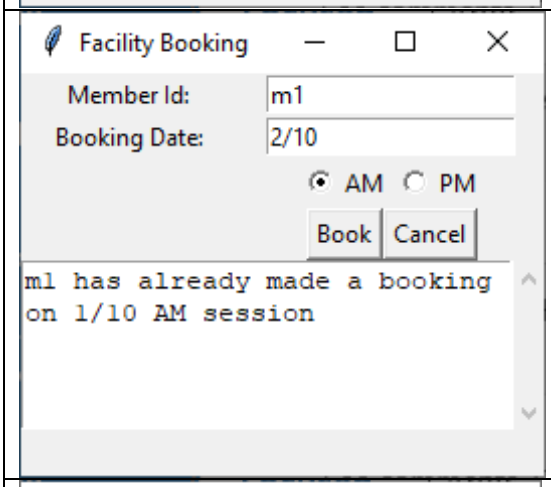
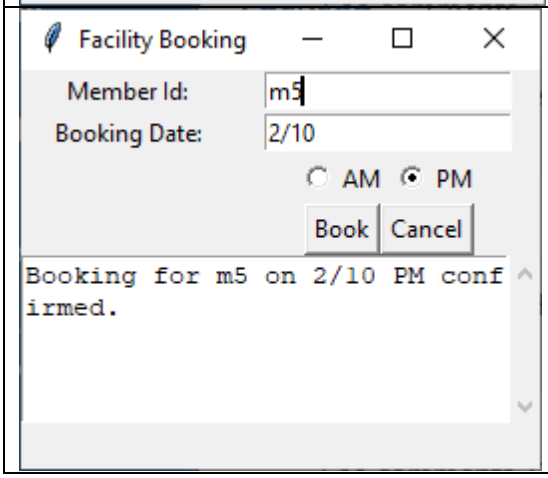
Indicate how this dictionary structure should be declared based on the code given. Indicate the line number where the statement should be inserted.

(4 marks)

Question 4d

Write and develop statements that will implement the event handler for Book button.

When a user clicks on the Book button, the booking is added to the dictionary if there are no errors. Below are demonstrations of the booking process:

	If button is clicked when Member Id or Booking Date is empty, display a message. For this prototype, booking date is of string type. There is no requirement to check for proper date format.
	A member can only make 1 booking per week. If there is an existing booking in the dictionary for this member, display a message.
	If the booking is successful, display a confirmation message.

(13 marks)

----- END OF PAPER -----